Практическое занятие № 8

1. Цель работы

Получить практические навыки шифрования и расшифрования конфиденциальных данных в PostgreSQL, изучить применение расширения `pgcrypto` и определить основные требования к безопасному хранению криптографических ключей.

2. Задачи работы

Студент должен:

1. Создать таблицу с персональными данными студентов.
2. Подключить расширение `pgcrypto`.
3. Выполнить шифрование отдельного поля с помощью `pgp_sym_encrypt`.
4. Проверить хранение зашифрованных данных.
5. Выполнить расшифрование с помощью `pgp_sym_decrypt`.
6. Сравнить открытое и зашифрованное хранение информации.
7. Проанализировать риски хранения ключей и предложить меры защиты.

3. Теоретические сведения

Шифрование — это преобразование открытых данных в шифротекст с использованием криптографического алгоритма и ключевого материала.

В PostgreSQL расширение `pgcrypto` предоставляет функции для выполнения различных криптографических операций.

В данной работе используются две функции:

- `pgp_sym_encrypt(data, password)` — шифрует данные с использованием парольной фразы.
- `pgp_sym_decrypt(data, password)` — расшифровывает данные при наличии правильной парольной фразы.

Для хранения зашифрованных значений используется тип `BYTEA`, предназначенный для двоичных данных.

Важно: парольная фраза, приведённая в работе, используется только в учебных целях. В реальной системе ключи необходимо хранить отдельно от базы данных, ограничивать доступ к ним и не записывать их в исходный код или журналы.

4. Ход выполнения работы

Этап 1. Подготовка базы данных

Откройте PostgreSQL через pgAdmin или другой SQL-клиент.

Создайте учебную таблицу:

```
CREATE TABLE students_security (
```

```
    id SERIAL PRIMARY KEY,
    student_name TEXT NOT NULL,
    document_number TEXT NOT NULL
);
```

Добавьте тестовые данные:

```
INSERT INTO students_security
    (student_name, document_number)
VALUES
    ('Иван Петров', 'DOC-1001'),
    ('Анна Сидорова', 'DOC-1002'),
    ('Алексей Смирнов', 'DOC-1003');
```

Выполните запрос:

```
SELECT * FROM students_security;
```

Задание: убедитесь, что в таблице присутствуют три записи. Объясните, почему хранение номеров документов в открытом виде может создавать риск утечки персональных данных.

Этап 2. Подключение расширения `pgcrypto`

Выполните команду:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Если команда завершилась успешно, расширение доступно в вашей учебной базе данных.

Если возникла ошибка, сообщите преподавателю: установка расширения может требовать дополнительных прав или настройки сервера.

Проверьте доступность функции шифрования:

```
SELECT encode(
    pgp_sym_encrypt(
        'TEST-123',
        'class-demo-key'
    ),
    'hex'
) AS encrypted_value;
```

Задание: зафиксируйте результат и объясните, почему полученное значение отличается от исходной строки `TEST-123`.

Этап 3. Создание таблицы для зашифрованных данных

Создайте новую таблицу:

```
CREATE TABLE students_encrypted (
    id SERIAL PRIMARY KEY,
    student_name TEXT NOT NULL,
    encrypted_document BYTEA NOT NULL
);
```

В отличие от первой таблицы, здесь номер документа будет храниться в зашифрованном виде.

Добавьте данные:

```
INSERT INTO students_encrypted
    (student_name, encrypted_document)
VALUES
    (
        'Иван Петров',
        pgp_sym_encrypt(
            'DOC-1001',
            'class-demo-key'
        )
    ),
    (
        'Анна Сидорова',
        pgp_sym_encrypt(
            'DOC-1002',
            'class-demo-key'
        )
    ),
    (
        'Алексей Смирнов',
        pgp_sym_encrypt(
            'DOC-1003',
            'class-demo-key'
        )
    );
```

Здесь `class-demo-key` — условная учебная парольная фраза. Не используйте её в реальных системах.

Проверьте содержимое таблицы:

```
SELECT
    id,
    student_name,
    encode(encrypted_document, 'hex') AS encrypted_value
FROM students_encrypted;
```

Задание: сравните содержимое столбца `encrypted_value` с открытыми номерами документов из первой таблицы. Сделайте вывод о различиях между открытым и зашифрованным хранением.

Этап 4. Расшифрование данных

Теперь восстановите исходные значения с помощью функции `pgp_sym_decrypt`.

```
SELECT
    id,
    student_name,
    pgp_sym_decrypt(
        encrypted_document,
        'class-demo-key'
    ) AS document_number
FROM students_encrypted;
```

Убедитесь, что исходные номера документов восстановлены правильно.

Затем выполните учебный тест с другой парольной фразой:

```
SELECT pgp_sym_decrypt(
    encrypted_document,
    'incorrect-demo-key'
)
FROM students_encrypted;
```

При использовании неверной парольной фразы расшифрование должно завершиться ошибкой.

Задание:

1. Зафиксируйте результат успешного расшифрования.
2. Объясните, почему для восстановления данных требуется правильная парольная фраза.
3. Объясните, какие последствия может иметь потеря ключа шифрования.
4. Объясните, почему нельзя хранить парольную фразу рядом с зашифрованными значениями.

Не используйте настоящие персональные данные и реальные секреты.

Этап 5. Сравнение способов хранения

Выполните два запроса:

```
SELECT
    student_name,
    document_number
FROM students_security;
SELECT
    student_name,
    encode(encrypted_document, 'hex') AS encrypted_document
FROM students_encrypted;
```

Сравните результаты.

Заполните таблицу:

Критерий	Открытое хранение	Шифрование отдельных полей
Читаемость значения непосредственно в таблице		
Возможность восстановить исходное значение		
Необходимость защищать ключ		
Последствия утечки таблицы		
Удобство обычного поиска по значению		

Ответьте на вопрос: достаточно ли шифрования отдельных полей, чтобы полностью защитить базу данных?

В ответе обязательно рассмотрите права доступа, безопасность приложения и хранение ключей.

Этап 6. Проектирование защиты ключей

Рассмотрите ситуацию.

Университет хранит персональные данные студентов в PostgreSQL. Номера документов зашифрованы. Однако разработчик записал парольную фразу непосредственно в SQL-запрос, а приложение работает под учётной записью с расширенными правами.

Определите не менее четырёх рисков и предложите меры защиты.

Заполните таблицу:

Риск	Возможные последствия	Предлагаемая мера защиты
Ключ записан в исходном коде		
Приложение имеет избыточные права		
Резервная копия не зашифрована		
Ключ потерян или скомпрометирован		
Расшифрованные данные попадают в логи		

При разработке решения учитывайте следующие принципы:

- хранить ключи отдельно от базы данных;
- предоставлять приложению только необходимые права;
- шифровать резервные копии и защищать доступ к ним;
- предусматривать безопасную ротацию и восстановление ключей;
- исключать запись секретов и конфиденциальных значений в журналы.

5. Контрольные вопросы

1. Что такое шифрование данных?
2. Чем открытый текст отличается от шифротекста?
3. Для чего применяется расширение `pgcrypto`?
4. Какую задачу решает функция `pgp_sym_encrypt`?
5. Для чего используется `pgp_sym_decrypt`?
6. Каково назначение типа `BYTEA` в PostgreSQL?
7. Почему для расшифрования требуется правильная парольная фраза?
8. Чем шифрование отличается от хеширования?
9. Почему парольную фразу нельзя хранить непосредственно в исходном коде?
10. Какие дополнительные меры защиты необходимо применять вместе с шифрованием?